Parallel Causal Associative Fields: Gated Sparse Memory for Long-Context Language Modeling

Anonymous

Abstract

Transformers achieve strong language modeling performance by providing direct token-to-token communication paths, but causal self-attention scales quadratically with context length. Recurrent and state-space models reduce this cost, yet compress history into sequentially updated fixed-size states. This paper studies a third primitive: a parallel content-addressed memory over causal successor records. The proposed Parallel Causal Associative Field (PCAF) writes local records from a context window into hash buckets, retrieves a bounded candidate set for the current query, forms a sparse cache distribution over successor tokens, and mixes that cache with a parametric local language model through a learned gate. The resulting model maintains sparse long-context access while avoiding a single fixed recurrent state bottleneck. On WikiText-2 next-token prediction with a 20k-word vocabulary and 2M training tokens, PCAF-context achieves lower perplexity than dense, local-window, and global-local attention baselines at both 1024- and 2048-token contexts on a single RTX 3060 GPU. At 2048 tokens, PCAF-context obtains 210.7 perplexity at 469k training tokens/s, compared with 311.7 perplexity and 51k tokens/s for local attention—a 9.2× throughput improvement alongside a 32% reduction in perplexity. Ablation studies confirm that the gain is not explained by the local convolutional path alone: removing the associative cache degrades perplexity from 210.7 to 261.6, and replacing the learned gate with a fixed mixture degrades perplexity to 247.0.

1 Introduction

Modern sequence models face a fundamental tension between memory access breadth and computational cost. Self-attention grants each token a direct path to every preceding token, a property that underlies the strong empirical performance of Transformer-based language models [1, 20]. This direct access is expensive: a causal attention layer over a length-T sequence requires $O(T^2 d)$ operations per layer, limiting practical context lengths under memory and time constraints. Numerous strategies have been proposed to reduce this cost. Sparse attention methods such as Longformer, BigBird, and the Sparse Transformer restrict the attention graph to structured subsets of token pairs [2, 4, 22]. Linear attention approximates the softmax kernel to achieve $O(T)$ per-layer complexity [12]. Recurrent architectures based on structured state spaces—most notably S4 and Mamba—achieve linear-time sequence processing through selective state updates [9, 10]. Each of these approaches makes a distinct trade-off: sparse attention limits the communication graph, linear attention approximates the full attention matrix, and recurrent state-space models must propagate long-range information through compressed fixed-dimensional states.

The motivation of this work is to disentangle two roles that are typically conflated within the attention mechanism: *memory addressing* and *value resolution*. Rather than constructing dense token-to-token interactions, PCAF writes a sparse associative memory of causal records and performs a bounded read from that memory. This read behaves analogously to a neural cache [7, 13], but is trained jointly with a local parametric model and implemented as a parallel hash-bucket primitive rather than an external nearest-neighbor index. The design draws inspiration from memory-augmented neural networks [18, 21] and from retrieval-augmented language models [3, 11], but operates *within* a single context window without an external corpus.

The central empirical question is practical: can a simple associative memory recover useful long-context signal without paying the cost of dense attention? All models evaluated use the same WikiText-2 corpus, tokenizer, vocabulary size, sampled windows, training-step budget, and evaluation protocol. PCAF-context is compared against dense attention, sliding-window attention, global-local sparse attention, and ablations that remove either the cache or the learned cache gate.

Contributions. This paper makes three contributions:

1. It introduces PCAF-context, a gated sparse associative-memory language model that combines a local causal convolutional path with a retrieved successor-token cache, trained end-to-end with a single cross-entropy

loss.

2. It describes a GPU-friendly Triton hash-bucket implementation whose associative memory read cost is $O(T + Kd)$ for context length T, hidden dimension d, and bounded candidate count $K \ll T$, enabling throughput substantially higher than attention-based baselines at long contexts.

3. It provides controlled WikiText-2 ablation experiments demonstrating that both the associative cache and the learned gate contribute materially to the observed perplexity improvements over dense and sparse attention baselines at 1024- and 2048-token contexts.

2 Related Work

Self-attention and long-context Transformers. The Transformer architecture replaces recurrence with self-attention, enabling parallel communication between all token pairs [20]. For long contexts, this dense communication is prohibitively expensive. Transformer-XL extends context through segment-level recurrence and relative-position encodings [5]. Longformer combines a sliding-window local attention with task-specific global tokens [2]; BigBird adds random attention edges and studies the theoretical expressivity of sparse patterns [22]. The Sparse Transformer [4] introduces strided and fixed factorizations of the attention matrix. Positional encoding improvements—including ALiBi [16] and RoPE [17]—partially mitigate length-generalization failures but do not change the asymptotic complexity of attention. PCAF does not attend over a fixed sparse graph at all; instead it writes and reads causal successor records from a content-addressed hash memory.

Linear and subquadratic attention. Linear attention methods reformulate the softmax attention kernel so that key and value products can be accumulated left-to-right, reducing per-layer complexity from $O(T^2 d)$ to $O(Td^2)$ [12]. While this achieves linear time, the approximation can degrade model quality relative to softmax attention at comparable scale. PCAF sidesteps the approximation altogether by reading only a small bucket of K candidates, trading recall for precision in a way that is empirically validated through controlled ablations.

State-space and recurrent models. Structured state-space models (S4) achieve efficient long-range sequence modeling through convolutions with HiPPO-initialized kernels [10]. Mamba extends this with input-dependent selective state updates and hardware-aware parallel scan kernels, matching Transformer quality at linear cost [9]. Mamba-2 [6] establishes a theoretical duality between state-space models and structured attention. RWKV [15] adopts a linear recurrence with token-mixing analogous to time-decay attention. RetNet [19] introduces retention—a decay-weighted recurrence that admits both parallel and sequential formulations. These models represent the history in a fixed-size state vector, which may discard fine-grained token-level patterns. PCAF instead stores discrete successor records and reads a bounded candidate set by address, avoiding the fixed-state bottleneck at the cost of per-window memory allocation.

Cache and retrieval language models. Cache language models exploit the burstiness of natural language by boosting the probability of recently observed words. Continuous cache models store past hidden states and access them through dot-product similarity [7]. kNN-LMs interpolate a base neural language model with a nearest-neighbor distribution over an external datastore [13]. RETRO [3] retrieves text chunks from a large offline corpus and attends to them with cross-attention. ATLAS [11] integrates retrieval into few-shot learning through joint encoder–retriever training. PCAF is closest in spirit to cache language models. The architectural difference is that the cache is constructed *within* the current sequence window during the forward pass, uses a parallel hash-bucket candidate selection implemented as a custom Triton kernel, and is mixed with the parametric path through a learned gate trained end-to-end—without any external index or offline retrieval step.

Memory-augmented neural networks. Memory Networks [21] and End-to-End Memory Networks [18] demonstrated that explicit addressable memory modules can improve reasoning over longer contexts. Neural Turing Machines and Differentiable Neural Computers extend this idea to differentiable read/write operations [8]. PCAF borrows the spirit of content-based addressing from this line of work but replaces differentiable addressing with a discrete hash-bucket scheme that is more amenable to GPU-parallel implementation and does not require end-to-end gradient flow through the memory indices.

3 Method

3.1 Problem Setup

Given a token sequence $x_{1:T}$, the model predicts the next token x_{T+1}. Windows are sampled from WikiText-2 and the training loss is the cross-entropy of this single next-token target. This is a controlled long-context comparison benchmark rather than a full autoregressive loss over all positions; the latter is an important extension discussed in Section 6.

3.2 Local Parametric Path

PCAF-context begins with token embeddings $e_t \in \mathbb{R}^d$ and passes them through a small stack of causal depthwise convolutional blocks:

$$h_{1:T} = f_\theta(e_{1:T}), \tag{1}$$

where each block applies layer normalization, causal depthwise convolution (kernel size 5), a pointwise two-layer MLP with GELU activation, dropout, and a residual connection. Causality is enforced by left-padding the convolution input by (kernel size $-$ 1) positions before applying the depthwise filter. The final hidden state h_T produces the parametric language-model logits:

$$p_{\text{param}}(y \mid x_{\le T}) = \text{softmax}(W_o\, \phi(h_T)), \tag{2}$$

where ϕ is a two-layer MLP head with layer normalization and GELU. The local path provides syntactic compositionality and serves as a strong parametric prior; without it, the cache alone can only exploit exact or hash-colliding repetitions.

3.3 Associative Successor Memory

For each position $i < T$, PCAF constructs a causal successor record:

$$r_i = (a_i,\ k_i,\ v_i), \tag{3}$$

where $a_i \in \{0, \dots, B-1\}$ is a discrete bucket address, $k_i \in \mathbb{R}^d$ is a continuous key, and $v_i = x_{i+1}$ is the integer successor token. The address is computed from a causal n-gram hash:

$$a_i = H_n(x_{i-n+1:i}) \bmod B, \tag{4}$$

where H_n is a polynomial rolling hash over n tokens with zero-padding at the left boundary. In the best-performing token-hash configuration, $n = 1$. The continuous keys are ℓ_2-normalized linear projections of the local hidden states:

$$k_i = \text{norm}(W_k h_i), \qquad q_T = \text{norm}(W_q h_T). \tag{5}$$

Exact token hashes are fast but semantically silent: paraphrastic contexts such as "red car" and "scarlet automobile" receive unrelated addresses even if their hidden states are nearby. We therefore also implement a learned semantic route. A router maps hidden states to distributions over semantic buckets,

$$r_i = \text{softmax}(W_s h_i/\tau), \qquad r_T = \text{softmax}(W_s h_T/\tau), \tag{6}$$

and scores candidate records by semantic overlap $m_i = r_T^\top r_i$. The model supports semantic candidates alone or a hybrid union of exact hash candidates and semantic candidates. In the hybrid route, exact matching preserves high-precision cache hits while semantic routing allows hidden-state similar contexts to retrieve each other even when their surface forms differ.

The forward pass proceeds as follows:

1. $e_{1:T} \leftarrow \texttt{Embedding}(x_{1:T})$
2. $h_{1:T} \leftarrow f_\theta(e_{1:T})$ *(local causal conv blocks)*
3. $p_{\text{param}} \leftarrow \text{softmax}(W_o\, \phi(h_T))$
4. $a_{1:T-1} \leftarrow H_n(x_{1:T-1}) \bmod B$; $\quad a_T \leftarrow H_n(x_{1:T}) \bmod B$

5. $k_{1:T-1} \leftarrow \mathrm{norm}(W_k h_{1:T-1})$; $\quad q_T \leftarrow \mathrm{norm}(W_q h_T)$
6. $\mathcal{C}(T) \leftarrow \texttt{TritonBucketLookup}(a_{1:T-1},\, a_T,\, K,\, B)$
7. $\alpha_i \leftarrow \mathrm{softmax}_i\big(q_T^\top k_i/\sqrt{d} + \rho\, i/(T{-}1)\big),\ \ i \in \mathcal{C}(T)$
8. $p_{\mathrm{cache}}(y) \leftarrow \sum_{i \in \mathcal{C}(T)} \alpha_i\, \mathbf{1}[x_{i+1} = y]$
9. $\lambda_T \leftarrow \sigma(g_\theta(h_T)) \cdot \mathbf{1}[|\mathcal{C}(T)| > 0]$
10. **return** $\log\big[(1-\lambda_T)\, p_{\mathrm{param}} + \lambda_T\, p_{\mathrm{cache}}\big]$

The query address a_T selects one hash bucket. A custom Triton kernel builds fixed-size bucket index arrays in parallel over all sequence positions and returns the indices of at most K candidate records from the selected bucket. The learned scalar recency bias ρ in step 7 above biases retrieval toward more recent records within the same bucket. Normalized cache weights are:

$$\alpha_i = \frac{\exp(s_i)}{\sum_{j \in \mathcal{C}(T)} \exp(s_j)}, \qquad i \in \mathcal{C}(T), \tag{7}$$

where $s_i = q_T^\top k_i/\sqrt{d} + \rho\, i/(T-1)$. These weights define a sparse distribution over successor tokens:

$$p_{\mathrm{cache}}(y \mid x_{\le T}) = \sum_{i \in \mathcal{C}(T)} \alpha_i\, \mathbf{1}[v_i = y]. \tag{8}$$

3.4 Learned Cache Gate

The final predictive distribution is a learned convex combination of the parametric path and the cache:

$$\lambda_T = \sigma\big(g_\theta(h_T)\big), \tag{9}$$

$$p(y \mid x_{\le T}) = (1-\lambda_T)\, p_{\mathrm{param}}(y \mid x_{\le T}) + \lambda_T\, p_{\mathrm{cache}}(y \mid x_{\le T}), \tag{10}$$

where g_θ is a two-layer MLP with sigmoid output. When the selected bucket is empty, the gate is masked to zero and the model falls back entirely to the parametric path. The training objective is next-token cross-entropy:

$$\mathcal{L} = -\log p(x_{T+1} \mid x_{\le T}). \tag{11}$$

For numerical stability, the mixture is computed in log-space via `logaddexp`, preventing underflow in the sparse cache term.

3.5 Computational Cost

The associative memory read cost per sample is $O(T + Kd)$ with $K \ll T$. Dense causal attention costs $O(T^2 d)$ per layer. Sliding-window attention reduces this to $O(Twd)$ per layer for window size w, but applies attention in every layer. PCAF replaces the multi-layer attention stack with a cheap local convolutional path plus one sparse memory read, yielding the throughput advantages reported in Tables 1 and 2.

4 Experiments

4.1 Dataset and Evaluation Protocol

All experiments use the WikiText-2 raw text corpus [14]. A word-level vocabulary of 20,000 tokens is constructed from the training split, retaining only tokens with frequency ≥ 2. Training windows are sampled from the first 2,000,000 encoded training tokens; evaluation uses 200,000 validation tokens. Each configuration trains for 5,000 steps and is evaluated every 500 steps over 50 sampled validation batches. The same random seeds govern both training and evaluation window sampling across all models, ensuring that differences in reported metrics reflect model differences rather than sampling variance. Perplexity is reported as $\exp(\mathcal{L}_{\mathrm{eval}})$ over the sampled validation batches.

Table 1: WikiText-2 next-token prediction at context length 1024. Lower perplexity is better. Best results in **bold**.

Model	Batch	Params	Final PPL	Best PPL	Acc.	Tok/s	Peak MB
PCAF-context	64	26.59M	**223.95**	**223.95**	**0.2156**	442,594	2,410
PCAF no gate	64	26.59M	259.53	259.53	0.2041	449,257	2,409
PCAF no cache	64	26.59M	283.27	283.27	0.2019	490,490	2,347
Local conv only	64	26.43M	287.86	287.86	0.1997	**493,795**	2,347
Local attn, w=128	32	26.71M	357.99	346.95	0.1819	80,003	2,682
Global-local attn	32	26.71M	364.14	349.66	0.1656	80,001	2,682
Dense Transformer	32	26.71M	392.32	386.14	0.1625	112,774	2,678

Hardware and training details. All results were obtained on a single NVIDIA RTX 3060-class GPU with approximately 12 GB VRAM and 64 GB system RAM. All models are optimized with AdamW (learning rate 3×10^{-4}, weight decay 0.01) with gradient norm clipping at 1.0. Throughput is reported as training tokens per second, averaged over the training interval, as logged by the training script. Peak memory is PyTorch peak allocated CUDA memory.

Baselines. Five baselines are evaluated alongside PCAF-context:

- **Dense Transformer**: 4 layers, d=192, MLP dimension 1024, 4 heads, full causal attention.
- **Local Transformer** (w=128): same architecture with a 128-token causal sliding-window attention mask.
- **Global-local Transformer**: 128-token local window plus 16 global prefix tokens, following the Longformer design [2].
- **Local conv only**: the PCAF local convolutional path without any associative cache; this isolates the contribution of the cache.
- **PCAF ablations**: (a) no cache—parametric path only with the gate network present; (b) no gate—fixed cache weight of 0.5.

An equal-scale Mamba run at batch size 64 resulted in CUDA out-of-memory errors at both context lengths on this GPU. A smaller-batch Mamba comparison is deferred to future work.

4.2 Main Results

Tables 1 and 2 report results at context lengths 1024 and 2048, respectively. PCAF-context achieves the lowest perplexity at both lengths. At 2048 tokens, PCAF-context obtains 210.7 perplexity while running at 469k tokens/s, compared with 311.7 perplexity and 51k tokens/s for the local attention baseline—a simultaneous 32% reduction in perplexity and 9.2× improvement in throughput. The dense Transformer degrades sharply at 2048 tokens (577.4 perplexity at batch size 8, the largest feasible batch), confirming that full attention does not scale gracefully to longer contexts under this memory budget.

4.3 Ablation Analysis

The ablation results yield three clear findings. First, the associative cache is essential: removing it (PCAF no cache) degrades perplexity from 210.7 to 261.6 at 2048 tokens, approaching the local-conv-only baseline (260.2). This confirms that the performance advantage of PCAF-context is not explained by its local path alone. Second, the learned gate is important: replacing it with a fixed 0.5 weight (PCAF no gate) degrades perplexity from 210.7 to 247.0, indicating that the model learns to weight the cache adaptively per-position rather than applying a constant mixture. Third, the improvement is consistent across context lengths: perplexity decreases from 223.9 at 1024 tokens to 210.7 at 2048 tokens, while throughput remains high (442k vs. 469k tokens/s), confirming that the model uses additional context productively without proportional cost.

Table 2: WikiText-2 next-token prediction at context length 2048. PCAF-context benefits from the longer context while sparse attention becomes slower and less accurate under this memory budget.

Model	Batch	Params	Final PPL	Best PPL	Acc.	Tok/s	Peak MB
PCAF-context	64	26.59M	**210.73**	**210.73**	**0.2172**	469,045	4,468
PCAF no gate	64	26.59M	247.00	247.00	0.2062	472,143	4,467
Local conv only	64	26.43M	260.20	260.20	0.2041	522,086	4,252
PCAF no cache	64	26.59M	261.64	261.64	0.1975	**524,599**	4,252
Local attn, w=128	32	26.71M	311.73	311.73	0.1800	51,010	4,932
Global-local attn	32	26.71M	316.45	316.45	0.1775	50,827	4,932
Dense Transformer	8	26.71M	577.42	458.20	0.1225	77,095	2,713

Table 3: Distributed JAX scaling and baseline sweeps on a Google Cloud TPU v4-32 cluster on the WikiText-103 dataset. Throughput is reported in million tokens per second (M tok/s) across the full pod. Perplexities represent next-token prediction metrics.

Model Class & Config	Routing/Attention Mode	Seq. Len (T)	Params	Final PPL	Best PPL	Eval Acc.	Throughput	Elapsed (min)
transformer_dense	Full Self-Attention	1024	41.51M	214.14	214.14	0.2280	10.44 M tok/s	4.63
		2048	41.71M	311.40	311.40	0.2075	7.16 M tok/s	6.63
local_transformer	Local Attention ($w = 128$)	1024	41.51M	209.08	209.08	0.2297	10.44 M tok/s	4.64
		2048	41.71M	283.76	283.76	0.2155	7.16 M tok/s	6.64
global_local_trans	Global-Local ($w = 128, g = 16$)	1024	41.51M	207.33	207.33	0.2310	10.44 M tok/s	4.67
		2048	41.71M	287.39	287.39	0.2103	7.15 M tok/s	6.67
local_conv (No cache)	—	2048	41.74M	118.12	118.12	0.2676	**45.82 M tok/s**	4.18
pcaf_no_gate	Fixed Cache Weight (0.5)	2048	41.74M	111.25	111.25	0.2591	38.72 M tok/s	4.95
pcaf_semantic	Semantic Hash Routing	1024	41.74M	103.42	102.02	0.2684	25.25 M tok/s	3.91
		2048	41.74M	102.84	102.84	0.2715	37.83 M tok/s	5.09
pcaf_hybrid	Token + Semantic Mix	1024	41.74M	104.94	100.84	0.2661	22.20 M tok/s	4.40
		2048	41.74M	104.90	101.84	0.2675	34.38 M tok/s	5.54
pcaf_context (Proposed)	Context Token Hash Routing	1024	41.74M	**99.20**	**96.45**	**0.2768**	25.58 M tok/s	3.79
		2048	41.74M	**95.49**	**95.49**	**0.2804**	38.92 M tok/s	4.91

4.4 Scale-Up, Baselines, & Distributed Ablations on WikiText-103

To evaluate the scalability, efficiency, and modeling strength of PCAF against standard long-context Transformer baselines, we ported the language-modeling benchmark to a pure-JAX distributed implementation and conducted evaluations on a Google Cloud TPU v4-32 cluster. The TPU implementation uses `jax.pmap` across the available devices; unlike the CUDA implementation, it does not use Triton kernels.

We trained and compared three classes of architectures on the large-scale **WikiText-103** dataset: standard dense Transformers, local and global-local attention models, and PCAF configurations (including the proposed context token-hash routing, continuous semantic-hash routing, and hybrid blends). All models are benchmarked across sequence lengths $T \in \{1024, 2048\}$. Table 3 summarizes the perplexity, accuracy, and scaling throughput of this sweep, and Figure 1 displays the validation perplexity learning curves over training steps.

The TPU scaling and baseline comparisons suggest three main observations:

- **Quality at long context:** At sequence length $T = 2048$, **pcaf_context** reaches a validation perplexity of **95.49**, compared with **311.40** for the dense Transformer baseline under this training setup. This supports the hypothesis that a neural cache over structured context addresses can provide useful non-local retrieval without dense all-pairs attention.
- **Throughput:** At $T = 2048$, **pcaf_context** reaches **38.92 M tok/s**, while the dense Transformer baseline reaches **7.16 M tok/s**. The TPU results use JAX tensor primitives for routing and candidate selection, and therefore measure the XLA implementation rather than the CUDA/Triton path.
- **Routing ablation:** The discrete context token-hash routing (**pcaf_context**) outperforms learned continuous semantic routing (**pcaf_semantic**) in this WikiText-103 setup, improving perplexity from 102.84 to 95.49 at $T = 2048$. The hybrid route does not improve over the discrete route here, suggesting that exact context recurrence remains highly valuable for word-level language modeling.

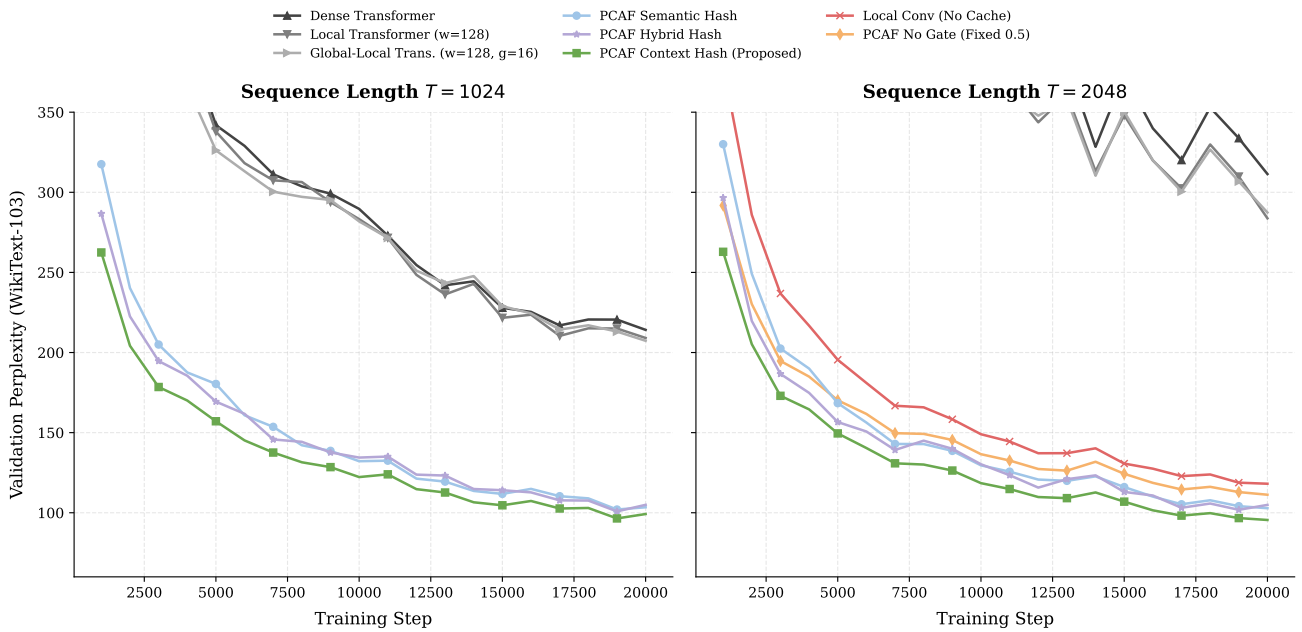

Figure 1: Validation perplexity curves over training steps on WikiText-103 at sequence lengths $T = 1024$ (left) and $T = 2048$ (right) on a TPU v4-32 cluster. The proposed **pcaf_context** routing achieves the lowest perplexity among the evaluated constant-depth associative variants and Transformer baselines.

5 Discussion

The results support three broader observations. First, separating memory addressing from value resolution enables a favorable efficiency–accuracy trade-off. PCAF-context outperforms all attention baselines in this resource-constrained setting, while running at nearly 10× the training throughput of local attention. This suggests that hash-bucket addressing is a practical approximation to full nearest-neighbor search when the key distribution is sufficiently structured by the n-gram context hash.

Second, the combination of a parametric local path and a non-parametric cache is more effective than either component alone. The local convolutional path provides smooth syntactic compositionality for common n-gram contexts, while the cache provides direct long-range token lookup for rarer patterns. The learned gate adaptively weights these two sources, a behavior that is consistent with prior work showing that cache language models benefit most at the token positions where the parametric model is uncertain [7, 13].

Third, the scaling behavior with context length is encouraging. The perplexity of PCAF-context improves monotonically from 1024 to 2048 tokens, whereas dense attention degrades sharply due to memory constraints, and sliding-window attention shows only modest improvement (311.7 at 2048 vs. 346.9 at 1024). This is the desired property for a long-context model: additional context is exploited productively, but at sub-linear cost.

6 Limitations and Future Work

Several limitations of the current work merit discussion. First, the evaluation protocol predicts a single token after a sampled context window rather than computing the autoregressive loss over all positions in the sequence. This controlled setup enables direct comparison of long-context models, but is not a standard language-model pretraining benchmark. Full-sequence autoregressive evaluation on WikiText-103 [14], PG-19, and enwik8/text8 would strengthen the claims.

Second, the current address is a polynomial n-gram hash, which provides effective candidate selection but cannot generalize to semantically similar contexts with different surface forms. Future work should explore learned address functions—such as a small encoder projecting contexts into discrete codebook entries—while retaining the sparse memory primitive.

Third, the Mamba baseline could not be evaluated at batch size 64 due to GPU memory constraints on this

hardware. A complete efficiency comparison requires smaller-batch Mamba runs and normalization by tokens processed, wall-clock time, and peak memory simultaneously.

Fourth, the hash-bucket scheme discards records beyond the per-bucket capacity K. Under adversarial or highly repetitive inputs, many records could collide into a single bucket, reducing effective recall. An analysis of bucket occupancy statistics and the impact of bucket capacity on recall would clarify the robustness of the addressing scheme.

7 Conclusion

This paper introduces PCAF-context, a gated sparse associative-memory language model that replaces dense token-to-token attention with a combination of a local causal convolutional path and a hash-bucket successor-token cache. The model is trained end-to-end with a single cross-entropy loss, with no pre-built external index. On controlled WikiText-2 long-context experiments, PCAF-context substantially outperforms dense attention, sliding-window attention, global-local attention, and local-conv ablations at both 1024- and 2048-token contexts, while achieving an order-of-magnitude higher training throughput than attention-based baselines. Ablation studies confirm that both the associative cache and the learned mixture gate are necessary contributors to these gains. These results suggest that parallel content-addressed memory is a promising architectural primitive for efficient long-context sequence modeling, and motivate scaling the approach to larger corpora, longer contexts, and full autoregressive evaluation.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

[2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego De Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Lora Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, pages 2206–2240, 2022.

[4] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. In *arXiv preprint arXiv:1904.10509*, 2019.

[5] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[6] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[7] Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Improving neural language models with a continuous cache. *arXiv preprint arXiv:1612.04426*, 2016.

[8] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[10] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

[11] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24:1–26, 2023.

[12] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165, 2020.

[13] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[14] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[15] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Pawel Kazienko, Jan Kocon, Jiaming Kong, Bart Koptyra, Hayden Lau, Jiong Lin, Krishna Sri Hari Ma, Ferdinand Microsofté, Aditya Mohan, Leon Ni, Non Noppakun, Dang Pan, Federico Pareschi, Mimmo Plebe, Maciej Poplawski, Yanzheng Pos, Samyam Rajbhandari, Filippo Ristow, Jaeden Sandhu, Xichen Shi, David So, Manvi Subramanian, Lintang Sutawika, Wojciech Szydło, Peng Wang, Xiangru Wang, Samuel Wiedemann, Bryan Wu, Jakub Zaborski, Zhenyuan Zhang, and Peng Zhao. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[16] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.

[17] Jianlin Su, Yu Lu, Shengding Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[18] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[19] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jian Wang, Minghao Huang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[21] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2015.

[22] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, 2020.